

Vertical Jump Challenge بازی: Speed Test Front-End

درجه سختی: ۷ از ۱۰

تکنولوژی مجاز: React یا Vue

نوع پروژه: بازی آرکید تعاملی شبیه Doodle Jump

زمان کل انجام پروژه: ۴ ساعت

هدف آزمون: پیاده‌سازی یک بازی پرشی عمودی به نام **Vertical Jump Challenge**. بازیکن باید شخصیت بازی را روی سکوها هدایت کند، امتیاز جمع کند، از موانع دوری کند و تا جای ممکن ارتفاع بیشتری بگیرد.

برای جلوگیری از وابستگی به برند یا کپی مستقیم، نام پروژه در این آزمون **Vertical Jump Challenge** است. سبک بازی مشابه بازی‌های پرش عمودی است، اما طراحی شخصیت، سکوها و ظاهر بازی باید توسط دانشجو انجام شود.

سناریوی بازی

در ابتدای برنامه، کاربر وارد صفحه شروع می‌شود و نام خود را وارد می‌کند. پس از شروع، کاربر وارد صفحه بازی می‌شود و یک شخصیت کوچک روی سکوها قرار دارد. شخصیت به‌صورت خودکار می‌پرد و کاربر باید با حرکت دادن شخصیت به چپ و راست، او را روی سکوهای بعدی فرود بیاورد.

با بالا رفتن شخصیت، صفحه بازی به سمت بالا اسکرول می‌شود و سکوهای جدید ظاهر می‌شوند. امتیاز کاربر بر اساس ارتفاع طی‌شده، جمع‌کردن آیتم‌ها و مدت زمان زنده ماندن محاسبه می‌شود. اگر شخصیت از پایین صفحه خارج شود یا با مانع خطرناک برخورد کند، بازی تمام می‌شود.

قوانین اصلی بازی

- شخصیت بازی همیشه تحت تأثیر جاذبه قرار دارد.
- وقتی شخصیت روی یک سکو فرود می‌آید، دوباره به سمت بالا پرتاب می‌شود.
- کاربر فقط حرکت افقی شخصیت را کنترل می‌کند.
- حرکت افقی باید با کیبورد یا دکمه‌های روی صفحه انجام شود.
- اگر شخصیت از سمت راست یا چپ صفحه خارج شود، می‌تواند از سمت مقابل وارد شود. این مورد اختیاری است اما امتیاز مثبت دارد.
- سکوها باید به‌صورت تصادفی تولید شوند.
- بعضی سکوها ثابت، بعضی شکسته‌شونده و بعضی متحرک هستند.
- اگر شخصیت از پایین صفحه خارج شود، بازی تمام می‌شود.
- بازی باید تایمر داشته باشد.
- در طول بازی باید امتیاز کاربر نمایش داده شود.

کنترل‌های بازی

عملیات	روش کنترل
حرکت به چپ	کلید ArrowLeft یا A
حرکت به راست	کلید ArrowRight یا D
شروع دوباره بازی	دکمه Restart
موبایل	دو دکمه لمسی چپ و راست روی صفحه

ورود اولیه کاربر

- در صفحه شروع، از کاربر نام دریافت شود.
- نام خالی قابل قبول نیست.
- نام کاربر باید در صفحه بازی و صفحه نتیجه نمایش داده شود.
- در صورت Refresh شدن صفحه، بهتر است نام کاربر حفظ شود. استفاده از LocalStorage مجاز است.

Routing / مسیرها

پروژه باید حداقل دارای دو مسیر باشد. استفاده از Router در React یا Vue الزامی است.

مسیر پیشنهادی	توضیح
/ یا welcome/	صفحه ورود نام کاربر، نمایش توضیح کوتاه و دکمه شروع بازی
game/	صفحه اصلی بازی، نمایش شخصیت، سکوها، امتیاز، تایمر و نام کاربر
result/	صفحه نتیجه نهایی، نمایش امتیاز، زمان زنده ماندن، ارتفاع، آیتمهای جمع شده و وضعیت ارسال نتیجه به API

حداقل دو مسیر الزامی است. مسیر نتیجه پیشنهاد می شود و امتیاز مثبت دارد.

منبع داده بازی

داده های بازی باید از یک فایل JSON آماده یا API ساده GET خوانده شوند. داده زیر به عنوان فایل آماده `jump-levels.json` در اختیار دانشجو قرار می گیرد. دانشجو نیازی به ساخت Backend ندارد.

نمونه فایل `jump-levels.json`

```
{
  "levels": [
    {
      "id": 1,
      "title": "Green Valley",
      "difficulty": 5,
      "worldTheme": "forest",
      "gameWidth": 420,
      "gameHeight": 620,
      "timeLimit": 180,

```

```
"gravity": 0.45,
"jumpPower": 11,
"playerSpeed": 5,
"platformCount": 12,
"platformMinGap": 55,
"platformMaxGap": 95,
"platformTypes": [
  {
    "type": "normal",
    "chance": 65,
    "score": 10
  },
  {
    "type": "moving",
    "chance": 20,
    "score": 20
  },
  {
    "type": "breakable",
    "chance": 15,
    "score": 25
  }
],
"collectibles": [
  {
    "type": "coin",
    "chance": 30,
    "score": 50
  },
  {
    "type": "spring",
    "chance": 10,
    "boost": 16,
    "score": 30
  }
],
"hazards": [
  {
    "type": "spike",
    "chance": 8,
    "penalty": 100
  }
]
},
{
  "id": 2,
  "title": "Neon Tower",
  "difficulty": 7,
  "worldTheme": "neon",
  "gameWidth": 420,
  "gameHeight": 620,
  "timeLimit": 160,
  "gravity": 0.5,
  "jumpPower": 11.5,
  "playerSpeed": 5.6,
  "platformCount": 14,
```

```
"platformMinGap": 65,
"platformMaxGap": 110,
"platformTypes": [
  {
    "type": "normal",
    "chance": 50,
    "score": 10
  },
  {
    "type": "moving",
    "chance": 30,
    "score": 25
  },
  {
    "type": "breakable",
    "chance": 20,
    "score": 35
  }
],
"collectibles": [
  {
    "type": "coin",
    "chance": 25,
    "score": 60
  },
  {
    "type": "spring",
    "chance": 12,
    "boost": 18,
    "score": 40
  },
  {
    "type": "shield",
    "chance": 5,
    "duration": 6,
    "score": 80
  }
],
"hazards": [
  {
    "type": "spike",
    "chance": 12,
    "penalty": 120
  },
  {
    "type": "enemy",
    "chance": 6,
    "penalty": 200
  }
]
},
{
  "id": 3,
  "title": "Broken Sky",
  "difficulty": 8,
  "worldTheme": "sky",
```

```
"gameWidth": 420,
"gameHeight": 620,
"timeLimit": 150,
"gravity": 0.55,
"jumpPower": 12,
"playerSpeed": 6,
"platformCount": 15,
"platformMinGap": 75,
"platformMaxGap": 120,
"platformTypes": [
  {
    "type": "normal",
    "chance": 40,
    "score": 10
  },
  {
    "type": "moving",
    "chance": 35,
    "score": 30
  },
  {
    "type": "breakable",
    "chance": 25,
    "score": 40
  }
],
"collectibles": [
  {
    "type": "coin",
    "chance": 25,
    "score": 70
  },
  {
    "type": "spring",
    "chance": 15,
    "boost": 20,
    "score": 50
  },
  {
    "type": "shield",
    "chance": 6,
    "duration": 5,
    "score": 90
  }
],
"hazards": [
  {
    "type": "spike",
    "chance": 15,
    "penalty": 130
  },
  {
    "type": "enemy",
    "chance": 8,
    "penalty": 220
  }
]
```

```
]
}
]
}
```

نحوه استفاده از داده‌ها

- در شروع هر بازی، یکی از Level ها به صورت تصادفی انتخاب شود.
- تنظیمات فیزیک بازی مانند جاذبه، قدرت پرش و سرعت بازیکن از Level انتخاب شده خوانده شود.
- سکوهاى اولیه بر اساس تعداد platformCount تولید شوند.
- نوع سکوها بر اساس مقدار chance در JSON تعیین شود.
- آیتم‌های قابل جمع‌آوری و موانع نیز به صورت تصادفی و بر اساس Chance تولید شوند.
- با بالا رفتن بازیکن، سکوهاى جدید باید به صورت تصادفی در بالای صفحه ساخته شوند.
- با Level، Restart، یا چینش سکوها باید دوباره تصادفی شود.

رندم‌سازی نباید باعث شود بازی از همان ابتدا غیرقابل انجام باشد. فاصله عمودی بین سکوها باید در محدوده قابل پرش باشد و از مقادیر platformMinGap و platformMaxGap استفاده کند.

انواع سکوها

نوع سکو	توضیح	رفتار مورد انتظار
normal	سکوی معمولی	بازیکن با برخورد از بالا روی آن می‌پرد.
moving	سکوی متحرک	به چپ و راست حرکت می‌کند و بازیکن باید زمان‌بندی درستی داشته باشد.
breakable	سکوی شکستنی	بعد از یک بار برخورد، از بین می‌رود یا دیگر قابل استفاده نیست.

آیتم‌ها و موانع

نوع	توضیح	اثر روی بازی
coin	سکه قابل جمع‌آوری	امتیاز اضافه می‌دهد.
spring	فنر پرشی	قدرت پرش بعدی را بیشتر می‌کند.
shield	سپر محافظ	برای چند ثانیه برخورد با مانع را بی‌اثر می‌کند.
spike	تیغ یا مانع خطرناک	اگر سپر فعال نباشد، امتیاز کم می‌کند یا بازی را تمام می‌کند.
enemy	دشمن متحرک یا ثابت	برخورد با آن باعث پایان بازی می‌شود، مگر اینکه سپر فعال باشد.

منطق فیزیک بازی

بازی باید حس پرش عمودی داشته باشد. لازم نیست فیزیک کاملاً حرفه‌ای باشد، اما موارد زیر باید رعایت شود:

- شخصیت دارای موقعیت افقی و عمودی باشد.
- شخصیت دارای سرعت عمودی باشد.
- جاذبه در هر فریم روی سرعت عمودی اثر بگذارد.
- وقتی شخصیت از بالا با سکو برخورد می‌کند، سرعت عمودی به مقدار پرش منفی یا رو به بالا تغییر کند.
- برخورد با سکو فقط زمانی معتبر است که شخصیت در حال پایین آمدن باشد.
- وقتی شخصیت از نیمه بالایی صفحه عبور می‌کند، محیط بازی باید به پایین حرکت کند تا حس بالا رفتن ایجاد شود.
- ارتفاع طی‌شده باید ثبت و در امتیاز استفاده شود.

تایمر

- بازی باید تایمر داشته باشد.
- زمان هر Level از مقدار `timeLimit` در JSON خوانده شود.
- تایمر از زمان مشخص‌شده به سمت صفر کم شود.
- اگر زمان به صفر برسد، بازی تمام شود.
- اگر بازیکن از پایین صفحه سقوط کند، بازی حتی قبل از پایان تایمر تمام شود.
- بعد از پایان بازی، تایمر متوقف شود.

سیستم امتیازدهی

بازی باید در طول اجرا امتیاز داشته باشد و امتیاز لحظه‌ای در صفحه بازی نمایش داده شود.

فرمول پیشنهادی امتیاز

- هر ۱۰ پیکسل ارتفاع جدید: ۱ امتیاز
- برخورد موفق با سکوی `normal`: امتیاز مربوط به همان نوع سکو از JSON
- برخورد موفق با سکوی `moving`: امتیاز بیشتر نسبت به سکوی معمولی
- استفاده موفق از سکوی `breakable`: امتیاز بیشتر، اما فقط یک بار
- جمع‌کردن `coin`: امتیاز مشخص‌شده در JSON
- جمع‌کردن `spring`: امتیاز مشخص‌شده در JSON و افزایش پرش
- جمع‌کردن `shield`: امتیاز مشخص‌شده در JSON و فعال‌شدن محافظ
- برخورد با مانع: کسر امتیاز بر اساس مقدار `penalty`
- امتیاز نهایی نباید کمتر از صفر شود.

مثال: اگر بازیکن ۲۴۰۰ پیکسل ارتفاع بگیرد، ۱۰ سکوی معمولی، ۴ سکوی متحرک، ۲ سکوی شکستنی، ۳ سکه و ۱ فنر جمع کند، امتیاز نهایی از مجموع ارتفاع، سکوها و آیتم‌ها محاسبه می‌شود.

شرایط پایان بازی

پایان با سقوط

اگر شخصیت از پایین صفحه خارج شود، بازی تمام می‌شود و نتیجه به صفحه نتیجه منتقل می‌شود.

پایان با زمان

اگر تایمر به صفر برسد، بازی تمام می‌شود و امتیاز نهایی ثبت می‌شود.

پایان با برخورد خطرناک

اگر بازیکن با دشمن یا مانع خطرناک برخورد کند و سپر فعال نباشد، بازی تمام می‌شود. در حالت ساده‌تر، برخورد با Spike می‌تواند فقط امتیاز کم کند، اما برخورد با Enemy بازی را تمام کند.

مدیریت State

مدیریت وضعیت می‌تواند با State محلی، Pinia، Redux، Context یا هر روش استاندارد دیگر انجام شود. انتخاب روش آزاد است، اما State باید واضح و قابل بررسی باشد.

State‌های ضروری

- نام کاربر
- Level انتخاب‌شده
- وضعیت بازی: آماده شروع، در حال اجرا، متوقف‌شده، پایان‌یافته
- موقعیت بازیکن
- سرعت افقی و عمودی بازیکن
- لیست سکوها
- لیست آیتم‌های قابل جمع‌آوری
- لیست موانع و دشمن‌ها
- ارتفاع طی‌شده
- بیشترین ارتفاع ثبت‌شده در همان بازی
- امتیاز فعلی
- تعداد سکه‌های جمع‌شده
- وضعیت فعال بودن Shield
- زمان باقی‌مانده
- علت پایان بازی
- وضعیت ارسال نتیجه به API

تعاملات کاربر

- کاربر بتواند با کیبورد شخصیت را به چپ و راست حرکت دهد.
- در موبایل، دو دکمه لمسی برای چپ و راست وجود داشته باشد.
- کاربر بتواند بازی را Restart کند.
- کاربر بتواند بعد از پایان بازی به صفحه شروع برگردد.
- در زمان پایان بازی، نتیجه به‌صورت واضح نمایش داده شود.
- در طول بازی، بازخورد برخورد با سکو، جمع‌آوری آیتم و برخورد با مانع مشخص باشد.

ارتباط با API خارجی

پروژه باید حداقل یک ارتباط با API خارجی داشته باشد. بعد از پایان بازی، نتیجه نهایی باید به API زیر ارسال شود:

API ارسال نتیجه:

POST <https://jsonplaceholder.typicode.com/posts>

Body پیشنهادی برای ارسال نتیجه

```
{
  "playerName": "Ali",
  "game": "Vertical Jump Challenge",
  "levelId": 3,
  "levelTitle": "Broken Sky",
  "difficulty": 8,
  "score": 2840,
  "height": 6200,
  "coins": 7,
  "usedSprings": 2,
  "platformHits": {
    "normal": 18,
    "moving": 9,
    "breakable": 4
  },
  "timeSpent": 112,
  "timeRemaining": 38,
  "endReason": "fall",
  "createdAt": "2026-08-04T12:30:00.000Z"
}
```

پاسخ نمونه API

```
{
  "playerName": "Ali",
  "game": "Vertical Jump Challenge",
  "levelId": 3,
  "levelTitle": "Broken Sky",
  "difficulty": 8,
  "score": 2840,
  "height": 6200,
  "coins": 7,
  "usedSprings": 2,
  "platformHits": {
    "normal": 18,
    "moving": 9,
    "breakable": 4
  },
  "timeSpent": 112,
  "timeRemaining": 38,
  "endReason": "fall",
  "createdAt": "2026-08-04T12:30:00.000Z",
}
```

"id": 101

}

رفتار مورد انتظار API

- بعد از پایان بازی، نتیجه به API ارسال شود.
- هنگام ارسال نتیجه، وضعیت Loading نمایش داده شود.
- در صورت موفقیت، پیام موفقیت نمایش داده شود.
- در صورت خطا، پیام خطا نمایش داده شود.
- در صورت خطا، امکان ارسال مجدد نتیجه وجود داشته باشد.
- نیازی نیست نتیجه واقعاً در سرور ذخیره شود؛ API معرفی شده برای شبیه سازی کافی است.

الزامات UI

- صفحه بازی باید یک محدوده مشخص برای Game Area داشته باشد.
- شخصیت بازیکن باید قابل تشخیص باشد.
- سکوها باید از نظر نوع، ظاهر متفاوت باشند.
- سکوی معمولی، متحرک و شکستنی باید با رنگ یا شکل متفاوت نمایش داده شوند.
- سکه، فنر، سپر و مانع باید قابل تشخیص باشند.
- نام کاربر، امتیاز، زمان باقی مانده و ارتفاع باید در صفحه بازی دیده شوند.
- در صفحه نتیجه، امتیاز نهایی، علت پایان بازی، زمان و ارتفاع نمایش داده شود.
- برنامه باید در دستکاپ قابل بازی باشد و برای موبایل نیز دکمه های کنترلی داشته باشد.

Randomization / تصادفی سازی

- در هر بار شروع بازی، تجربه بازی باید متفاوت باشد.
- Level باید به صورت تصادفی از JSON انتخاب شود.
 - موقعیت افقی سکوها تصادفی باشد.
 - فاصله عمودی بین سکوها تصادفی اما در محدوده مجاز باشد.
 - نوع سکوها بر اساس chance تصادفی انتخاب شود.
 - آیتمها و موانع بر اساس chance تصادفی تولید شوند.
 - با بالا رفتن بازیکن، سکوها ی جدید به صورت تصادفی تولید شوند.

نکته مهم: تصادفی سازی نباید بازی را غیرممکن کند. اگر فاصله سکوها بیشتر از توان پرش بازیکن باشد، بازی غیرمنصفانه می شود.

خروجی مورد انتظار پروژه

۱. صفحه ورود نام کاربر

۲. حداقل دو Route

۳. خواندن داده از JSON یا API ساده GET
۴. انتخاب تصادفی Level
۵. تولید تصادفی سکوها، آیتمها و موانع
۶. نمایش Game Area
۷. حرکت افقی شخصیت با کیبورد
۸. دکمه‌های کنترلی برای موبایل
۹. پرش خودکار شخصیت بعد از برخورد با سکو
۱۰. تشخیص برخورد با سکوها
۱۱. تشخیص جمع‌آوری آیتمها
۱۲. تشخیص برخورد با موانع
۱۳. سیستم امتیازدهی
۱۴. نمایش ارتفاع طی‌شده
۱۵. تایمر فعال
۱۶. تشخیص پایان بازی
۱۷. ارسال نتیجه نهایی به API خارجی
۱۸. نمایش صفحه نتیجه

نکات مهم ارزیابی

بخش	انتظار
منطق بازی	حرکت، جاذبه، پرش، برخورد با سکو و پایان بازی درست پیاده‌سازی شده باشد.
State Management	مدیریت تمیز وضعیت بازیکن، سکوها، آیتمها، امتیاز و تایمر.
Randomization	تولید تصادفی Level، سکوها، آیتمها و موانع به شکل قابل بازی.
UI/UX	بازی قابل فهم، روان، خوانا و دارای بازخورد بصری مناسب باشد.
Routing	حداقل دو مسیر کاربردی وجود داشته باشد.
API	ارسال نتیجه با مدیریت Success، Loading و Error انجام شود.
کنترل‌ها	کنترل با کیبورد و حداقل پشتیبانی ساده از موبایل وجود داشته باشد.
کیفیت کد	کامپوننت‌بندی مناسب، نام‌گذاری واضح و کد قابل بررسی.

محدودیت‌ها

- استفاده از React یا Vue آزاد است.
- استفاده از TypeScript اختیاری است.
- استفاده از UI Library مانند MUI، Bootstrap، Tailwind یا Vuetify مجاز نیست.

- استفاده از Canvas مجاز نیست اما الزامی نیست.
- پیاده‌سازی با HTML و CSS معمولی نیز قابل قبول است، اگر رفتار بازی درست باشد.
- استفاده از موتور بازی‌سازی یا کتابخانه آماده بازی مجاز نیست.
- منطق اصلی حرکت، برخورد، پرش، تولید سکو و امتیازدهی باید توسط دانشجو پیاده‌سازی شود.

اگر پیاده‌سازی همه آیتم‌ها زمان‌بر شد، حداقل باید سکوهای معمولی، سکوهای متحرک، امتیازدهی، تایمر، سقوط، Restart و ارسال نتیجه به API کامل پیاده‌سازی شوند. برای کسب امتیاز کامل، سکوهای شکستنی، آیتم‌های جمع‌آوری‌شدنی و موانع نیز باید وجود داشته باشند.

تحويل نهایی

- لینک Repository یا فایل فشرده پروژه
- راهنمای کوتاه اجرای پروژه
- ذکر تکنولوژی استفاده‌شده: React یا Vue
- قرار دادن فایل JSON داده‌ها داخل پروژه، در صورت استفاده از فایل محلی
- توضیح کوتاه درباره منطق برخورد، تولید سکو و محاسبه امتیاز

زمان کل: شما ۲:۳۰ ساعت برای پیاده‌سازی این پروژه فرصت دارید. زمان‌بندی جداگانه برای بخش‌ها داده نمی‌شود و مدیریت زمان بر عهده شماست.